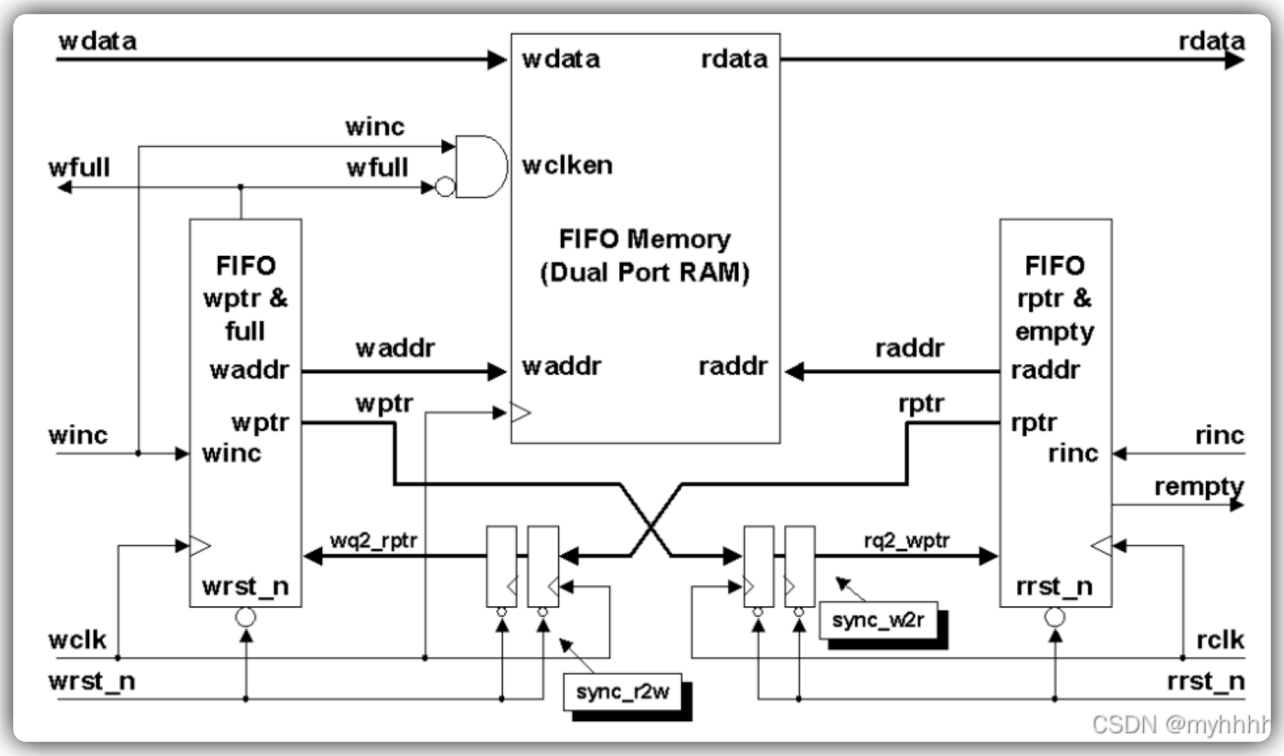
module 15: 同步FIFO

1. 同步FIFO

- 使用verilog，完成同步FIFO的设计，其中数据位宽为8位，FIFO的深度为16，其中：
  - 输入端口为
    - clk
    - rst_n（复位信号）
    - write_en（写使能）
    - read_en（读使能）
    - data_in，
  - 输出端口为
    - empty（空信号）
    - full（满信号）
    - data_out

2. 同步FIFO原理

- 1.同步FIFO和异步FIFO的结构差异



  - 以上是异步FIFO的结构图，可以发现，对于异步FIFO来说，它由几部分组成，最中间的是一块双口RAM，RAM的左边是写控制模块， 右边是读控制模块，下面的两级寄存器起到了格雷码状态下的地址信号跨时钟域作用，而对于

同步FIFO来说，因为读写共用同一个时钟频率，因此不需要寄存器同步的那一部分。为了最大程度的贴近于异步FIFO的设计，作者依旧采用格雷码的比较来产生空满信号，当然，使用一个计数器来判断空满信号也是可行的

- 在格雷码转二进制中，我们已经讨论了gray_code=binary_code^(binary_code >> 1)这种格雷码的产生方式的限制，即只适用于 2^N 的情况，这里对FIFO的深度约束依旧成立，假如我们设计一个14位深度的fifo，地址由14跳回1不满足单bit变化，会产生采样错误的问题，因此本篇我们依旧对偶数情况的格雷码按下不表，只讨论2的N次方的深度问题

- 设置读指针与写指针，在格雷码的情况下，若读写指针相等，证明FIFO为空，这个很好理解，那么什么情况下，FIFO为满呢？写指针转了一圈，追上了读指针的时候，FIFO应该是满状态，这里如果拿格雷码表格来举例的话，假设FIFO的深度为8位，只需要3位二进制编码就能表示，我们多使用一位，用4位的二进制编码来额外加上写指针追读指针的情况，那么以下的截图，读写指针为1+9，2+10，3+11以此类推的情况都是应该产生满信号的情况，找下规律，发现对于4bit格雷码来说，前两位相反，之后的位相同，就是verilog逻辑中判断满信号产生的条件

Decimal	Binary	Gray	Decimal of Gray
0	0000	0000	0
1	0001	0001	1
2	0010	0011	3
3	0011	0010	2
4	0100	0110	6
5	0101	0111	7
6	0110	0101	5
7	0111	0100	4
8	1000	1100	12
9	1001	1101	13
10	1010	1111	15
11	1011	1110	14
12	1100	1010	10
13	1101	1011	11
14	1110	1001	9
15	1111	1000	8

CSDN@SDNh@myhnh

- 并不是所有的寄存器块都进行了复位的，比如说描述data_in所在的块，作者没有进行复位，但这并不影响FIFO的正常工作，这是因为我们设计追寻的原则为“控制通路必须包含复位”，数据通路“是否复位可以选择”，即读写指针所在的通路rst_n到来的时候都需要复位，但data_in作为数据通路可以选择不带复位信号

3. RTL

```
module sync_fifo #(
    parameter   DATA_WIDTH = 8,
    parameter   DATA_DEPTH = 128 //2'b1000_0000,127-> 2'b0111_1111
)
(
    input  wire                 i_sys_clk           ,
    input  wire                 i_sys_rst_n          ,
    input  wire                 i_wren              ,
    input  wire                 i_rden              ,
```

```
    input  wire       [DATA_WIDTH-1:0] i_wdata                ,
    output wire       [DATA_WIDTH-1:0] o_rdata                ,
    output wire                        o_full                 ,
    output wire                        o_empty
);

    //定义ram的深度
    reg  [DATA_WIDTH-1:0]    mem_ram   [0:DATA_DEPTH-1];
    //读写fifo指针
    reg  [clogb2(DATA_DEPTH-1)-1 :0]    r_wr_ptr     ;
    reg  [clogb2(DATA_DEPTH-1)-1 :0]    r_rd_ptr     ;
    reg  [clogb2(DATA_DEPTH-1)   :0]    r_fifo_number ;

always@(posedge i_sys_clk or negedge i_sys_rst_n)
begin
    if(!i_sys_rst_n)
        r_wr_ptr <= 'd0;
    else if(i_wren && !o_full)
        r_wr_ptr <= r_wr_ptr + 1'b1;
    else
        r_wr_ptr <= r_wr_ptr ;
end

always@(posedge i_sys_clk or negedge i_sys_rst_n)
begin
    if(!i_sys_rst_n)
        r_rd_ptr <= 3'b0;
    else if(i_rden && !o_empty)
        r_rd_ptr <= r_rd_ptr + 3'b1;
    else
        r_rd_ptr <= r_rd_ptr ;
end


reg [DATA_WIDTH-1:0]      r_rdata  ;
always@(posedge i_sys_clk or negedge i_sys_rst_n)
begin
    if(!i_sys_rst_n)
        r_rdata <= 'd0;
    else if(i_rden && !o_empty)
        r_rdata <= mem_ram[r_rd_ptr];
end

integer  i  ;
always@(posedge i_sys_clk or negedge i_sys_rst_n)
begin
    if(!i_sys_rst_n)
    begin
        for(i = 0; i < DATA_DEPTH; i = i + 1)
        begin
            mem_ram[i] <= 'd0;
        end
    end
    else if(i_wren && !o_full)
        mem_ram[r_wr_ptr] <= i_wdata;
end

//fifo number cnt
always@(posedge i_sys_clk or negedge i_sys_rst_n)
begin
    if(!i_sys_rst_n)
        r_fifo_number <= 'd0 ;
    else if(i_wren  && i_rden  && !o_full && !o_empty)
        r_fifo_number <= r_fifo_number;
    else if(i_wren  && !i_rden && !o_full)
        r_fifo_number <= r_fifo_number + 1'b1;
    else if(!i_wren && i_rden  && !o_empty)
        r_fifo_number <= r_fifo_number - 1'b1;
    else
        r_fifo_number <= r_fifo_number;
end

assign  o_rdata = r_rdata ;
assign  o_full  = (r_fifo_number == DATA_DEPTH) ;
assign  o_empty = (r_fifo_number == 4'd0      ) ;
```

```
//7  2'b111  3
function integer clogb2(input integer number);
begin
    for(clogb2 = 0 ; number > 0 ; clogb2 = clogb2 + 1 )
        number = number >> 1;
end
endfunction

endmodule
```